Fixed NULL Tokens for Arithmetic Learning: A Compute-Matched Evaluation

Harvin Chhabra
harvinchhabra@gmail.com

October 2026

Abstract. Additional sequence positions offer a simple way to allocate computation before a language model commits to an answer. We study this approach in a 95K-parameter causal Transformer trained for integer addition, inserting four learned NULL tokens before the answer with answer-only supervision. NULL achieves 98.41% exact-answer accuracy, compared with 95.48% for the original baseline and 96.57% for a baseline trained from scratch at approximately the same training FLOPs. The compute-matched comparison yields a 1.84 percentage-point gain and 53.6% fewer exact-answer errors on 10,000 test examples. Answer-token cross-entropy also decreases from 0.0259 to 0.0128. Input ablations and longer-operand evaluations characterize the scope of this effect. The results suggest that, within the tested distribution, allocating computation through additional token positions can be more effective than extending conventional baseline training.

1 Introduction

Intermediate tokens expand the representations available to a causal Transformer before it generates an answer. Explicit scratchpads assign these tokens readable computational steps; an alternative is to provide positions without prescribing their semantic content. The motivation for this project was to examine whether otherwise uninformative tokens could serve as a useful computational workspace without written chain-of-thought supervision.

We investigate a minimal setting: character-level addition with a small Transformer. The NULL variant receives four fixed tokens between the problem and answer. We compare it with both an equal-duration baseline and a baseline given additional training to match the estimated NULL compute budget. This separates a simple accuracy comparison from the question of whether spending that training budget on a conventional model recovers the gain.

The main result is a 1.84 percentage-point accuracy advantage over the compute-matched baseline, corresponding to 184 additional correct answers per 10,000 problems. We also evaluate dependence on the inserted tokens, carry-count subgroups, and transfer to longer operands.

2 Related Work

Goyal et al. [1] introduce learnable pause tokens appended to language model prefixes, with output extraction delayed until after the pauses. Our fixed-NULL construction follows this general approach in a small arithmetic setting, with an additional comparison against extended baseline training under reconstructed training-compute budgets.

Pfau, Merrill, and Bowman [2] show that meaningless filler tokens can support computation on algorithmic tasks under suitable supervision. Nye et al. [3] instead train language models to emit explicit intermediate scratchpad steps, including for addition. Together, these studies motivate examining the contribution of intermediate positions separately from written reasoning content. Here, only answer characters and EOS are supervised.

Coconut [4] explores continuous latent computation by feeding hidden states back as subsequent input embeddings. In our implementation, each NULL position receives a learned discrete-token embedding plus a positional embedding. This provides a complementary, simpler construction using standard causal Transformer inference.

3 Experimental Setup

Architecture and token insertion. Both models use two Transformer layers, width 72, four attention heads, feedforward width 144, learned absolute positions, context length 128, no dropout, and tied token/output embeddings. The baseline has 95,256 parameters; adding NULL increases this to 95,328. Arithmetic is character-tokenized, including spaces. The NULL prompt takes the form

$$\texttt{12345 + 67890 = <NULL>}^{4} \rightarrow \texttt{80235 <EOS>}.$$

Four NULLs are inserted automatically after the complete problem prompt. The model does not choose their number. NULL IDs are not supervised targets; their representations receive gradients through the answer loss. Greedy answer decoding forbids NULL, PAD, and UNK outputs.

Training. Original runs use 1,000,000 training rows, five epochs, batch size 64, AdamW with learning rate

Model	Training steps	Training TFLOPs
Baseline	78,125	59.827680
NULL ×4	78,125	71.660160
Matched baseline	93,576	71.659968

Table 1: Estimated training budgets. TFLOPs denotes 10^{12} operations. The matched baseline processes 5,988,864 examples, versus 5,000,000 for each original run.

Model	Acc. (%)	Loss	Errors
Baseline	95.48	0.0329	452
Matched baseline	96.57	0.0259	343
NULL ×4	**98.41**	**0.0128**	**159**

Table 2: Original-distribution evaluation on 10,000 examples. Accuracy is exact-answer accuracy; loss is answer-token cross-entropy.

3×10^{-4}, gradient clipping at 1, and seed 42. Original operands range from 0 to 99,999. All evaluations reuse the same seed-42 sample of 10,000 rows from the million-row test file. Separate datasets contain 10,000 unique ordered pairs of exactly six-digit and seven-digit operands, generated with seeds 4206 and 4207 and Python-integer targets.

The primary metric is greedy exact-answer accuracy through EOS, with a 24-token generation cap and non-termination counted as incorrect. Answer-token cross-entropy is token-weighted, includes EOS, and excludes the prompt and inserted tokens. Each checkpoint uses its saved vocabulary.

Compute matching. For batch size B, padded length L, width d, layer count N, and vocabulary size V, we estimate dense matrix-multiplication FLOPs as

$$C_{\text{fwd}} = B\big[N(16Ld^2 + 4L^2d) + 2LdV\big], \qquad (1)$$
$$C_{\text{train}} \approx 3C_{\text{fwd}}.$$

A multiply-add counts as two FLOPs. The terms cover attention projections, feedforward layers of width $2d$, attention scores and weighted values, and the output projection. We reconstruct the original batch order and maximum padded lengths: 22 for baseline and 26 for NULL. Estimated step costs are 0.765794 and 0.917250 billion FLOPs, respectively, a 19.78% difference.

The matched baseline is trained from scratch using the original seed, initialization procedure, optimizer, learning rate, clipping, and shuffling. Only its duration is extended. The target is the original five-epoch NULL budget, matched to the nearest complete optimizer step (Table 1). The resulting mismatch is −0.000268%. During its first five epochs, the fresh baseline reproduces all 160 original loss-log entries within their recorded four-decimal precision.

4 Results

4.1 Compute-matched accuracy

Table 2 shows that extended baseline training improves accuracy from 95.48% to 96.57%. NULL achieves 98.41%, retaining a 1.84 percentage-point advantage at approximately equal training compute. Its answer-token loss is 0.0128, compared with 0.0259 for the matched baseline. Thus, both exact-answer accuracy and predictive loss favor the NULL model under the matched budget.

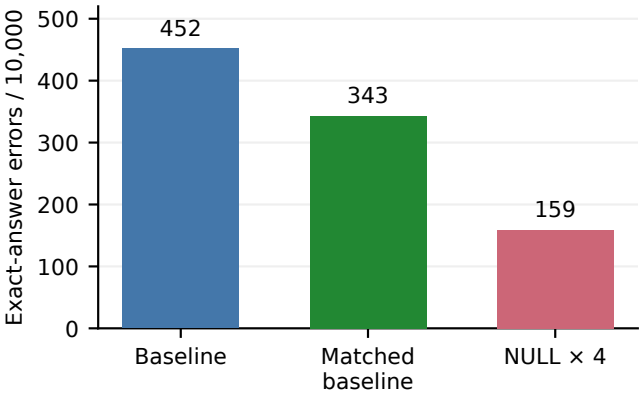


Figure 1: Exact-answer errors on the original-distribution test sample (lower is better). NULL makes 184 fewer errors than the compute-matched baseline, a relative reduction of 53.6%.

The absolute accuracy difference represents a substantial change in residual errors: NULL makes 159 errors instead of 343, a 53.6% reduction relative to the matched baseline (Figure 1). Additional baseline training therefore recovers part of the original gap, while the remaining difference supports further investigation of how computation is allocated across sequence positions. Carry-stratified results also favor NULL over the original baseline in every observed carry-count group (0–5).

4.2 Dependence on the inserted tokens

Inference-only interventions preserve the NULL checkpoint and change its prompt tokens (Table 3). Four NULLs achieve 98.41% accuracy; counts of 0, 1, 2, 3, or 8 yield 0%. Replacing the four NULLs with UNK while preserving length yields 1.52%. These results establish a strong dependence on the trained token configuration and motivate controls trained with alternative tokens or variable counts.

4.3 Operand length and inference cost

On both six- and seven-digit datasets, all three models score 0% exact-answer accuracy. The accuracy advantage is therefore specific to the original operand range in these evaluations.

NULL-checkpoint input	Accuracy (%)
NULL ×4 (trained count)	98.41
NULL ×{0, 1, 2, 3, 8}	0.00
UNK ×4 (same length)	1.52

Table 3: Original-distribution input ablations without retraining. Each count in the grouped row was evaluated separately.

For original-distribution full-prefix generation without a KV cache, estimated inference cost is 21.60 million FLOPs per problem for either baseline and 26.61 million for NULL (23.2% more). This counts active generation through EOS, excluding work on already-finished batch rows.

5 Limitations

Replication and scope. The study uses one initialization seed, one small architecture, and one task. The observed accuracy and error reductions are descriptive; robustness across seeds has not been established. Identical seeds also do not imply identical shared-weight initializations when vocabulary sizes differ. Larger models, additional tasks, and repeated runs are needed to assess transferability.

Mechanism and input distribution. The experiment does not identify the internal arithmetic procedure or test the faithfulness of visible chains of thought. Four extra positions are processed by the same two-layer network, rather than adding four layers or guaranteeing four serial reasoning iterations. NULL allocation is externally fixed; adaptive generation of thinking tokens is outside the present setup. Ablating counts changes answer positions, while UNK replacement introduces an untrained input pattern. These interventions cannot independently identify embedding, positional, and computational effects.

Length extrapolation. No model generalizes successfully to the tested longer operands. Learned absolute positions beyond the original training lengths are a possible contributor, alongside changes in arithmetic length and token-position roles. The current experiments do not distinguish these explanations. Length curricula and positional-encoding comparisons would provide useful controls.

Compute estimates. Equation 1 counts full-square attention and approximates backward cost as twice forward cost. It excludes normalization, softmax, activations, biases, embedding lookup, optimizer operations, and memory traffic. Matching this estimate does not match hardware time, energy, inference cost, or all architectural differences. Inference estimates can also appear smaller for OOD answers that terminate early, so they should be read with accuracy.

Data and evaluation coverage. Original train/test files share 97 exact expressions, or 187 test rows when operand reversal is allowed. None of the exact overlaps occurs in the 10,000-row evaluation sample; reversed-pair overlap for this sample was not separately reported. The original files contain duplicate rows. New longer operand sets have disjoint ranges from training, but cover only two lengths and one dataset seed per length.

6 Conclusion

Four fixed NULL tokens improve exact-answer accuracy in this small addition model, reaching 98.41% against 96.57% for a compute-matched baseline. The 1.84 percentage-point gain corresponds to 53.6% fewer errors and is supported by lower answer-token loss. The result motivates studying token-based allocation of computation alongside conventional increases in training duration. Multi-seed replications, trained token controls, and adaptive NULL-count policies are natural next steps.

References

[1] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. *Think before you speak: Training Language Models With Pause Tokens*. ICLR, 2024; preprint, 2023. arXiv:2310.02226.

[2] Jacob Pfau, William Merrill, and Samuel R. Bowman. *Let's Think Dot by Dot: Hidden Computation in Transformer Language Models*. 2024. arXiv:2404.15758.

[3] Maxwell Nye et al. *Show Your Work: Scratchpads for Intermediate Computation with Language Models*. 2021. arXiv:2112.00114.

[4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. *Training Large Language Models to Reason in a Continuous Latent Space*. Preprint, 2024; COLM, 2025. arXiv:2412.06769.

Reproducibility. Source code, metrics, dataset audits, and compute-method reports accompany this paper in the repository. The paper uses the saved experiment outputs; no new training or evaluation was performed for this revision.